

Competidor(a): \_\_\_\_\_

Número de inscrição: \_\_\_\_\_-\_\_\_\_\_ (opcional)

*Este Caderno de Tarefas não pode ser levado para casa após a prova. Após a prova entregue este Caderno de Tarefas para seu professor guardar. Os professores poderão devolver os Cadernos de Tarefas aos competidores após o término do período de aplicação das provas (12 de setembro de 2025).*



Olimpíada Brasileira de Informática

OBI2025

## Caderno de Tarefas

Modalidade Programação • Nível 2 • Competição Feminina

12 de setembro de 2025

A PROVA TEM DURAÇÃO DE 3 HORAS

Promoção:



Sociedade Brasileira de Computação

Apoio:



Coordenação:



# Instruções

LEIA ATENTAMENTE ESTAS INSTRUÇÕES ANTES DE INICIAR A PROVA

- Este caderno de tarefas é composto por 11 páginas (não contando a folha de rosto), numeradas de 1 a 11. Verifique se o caderno está completo.
- A prova deve ser feita individualmente.
- É proibido consultar a Internet, livros, anotações ou qualquer outro material durante a prova. É permitida a consulta ao *help* do ambiente de programação se este estiver disponível.
- As tarefas têm o mesmo valor na correção.
- A correção é automatizada, portanto siga atentamente as exigências da tarefa quanto ao formato da entrada e saída de seu programa; em particular, seu programa não deve escrever frases como “Digite o dado de entrada:” ou similares.
- Não implemente nenhum recurso gráfico nas suas soluções (janelas, menus, etc.), nem utilize qualquer rotina para limpar a tela ou posicionar o cursor.
- As tarefas **não** estão necessariamente ordenadas, neste caderno, por ordem de dificuldade; procure resolver primeiro as questões mais fáceis.
- Preste muita atenção no nome dos arquivos fonte indicados nas tarefas. Soluções na linguagem C devem ser arquivos com sufixo *.c*; soluções na linguagem C++ devem ser arquivos com sufixo *.cc* ou *.cpp*; soluções na linguagem Pascal devem ser arquivos com sufixo *.pas*; soluções na linguagem Java devem ser arquivos com sufixo *.java* e a classe principal deve ter o mesmo nome do arquivo fonte; soluções na linguagem Python 3 devem ser arquivos com sufixo *.py*; e soluções na linguagem Javascript devem ter arquivos com sufixo *.js*.
- Na linguagem Java, **não** use o comando *package*, e note que o nome de sua classe principal deve usar somente letras minúsculas (o mesmo nome do arquivo indicado nas tarefas).
- Para tarefas diferentes você pode escolher trabalhar com linguagens diferentes, mas apenas uma solução, em uma única linguagem, deve ser submetida para cada tarefa.
- Ao final da prova, para cada solução que você queira submeter para correção, copie o arquivo fonte para o seu diretório de trabalho ou pen-drive, conforme especificado pelo seu professor.
- Não utilize arquivos para entrada ou saída. Todos os dados devem ser lidos da entrada padrão (normalmente é o teclado) e escritos na saída padrão (normalmente é a tela). Utilize as funções padrão para entrada e saída de dados:
  - em Pascal: *readln*, *read*, *writeln*, *write*;
  - em C: *scanf*, *getchar*, *printf*, *putchar*;
  - em C++: as mesmas de C ou os objetos *cout* e *cin*.
  - em Java: qualquer classe ou função padrão, como por exemplo *Scanner*, *BufferedReader*, *BufferedWriter* e *System.out.println*
  - em Python: *read*, *readline*, *readlines*, *input*, *print*, *write*
  - em Javascript: *scanf*, *printf*
- Procure resolver a tarefa de maneira eficiente. Na correção, eficiência também será levada em conta. As soluções serão testadas com outras entradas além das apresentadas como exemplo nas tarefas.

# Copa do Mundo

Nome do arquivo: `copa.c`, `copa.cpp`, `copa.pas`, `copa.java`, `copa.js` ou `copa.py`

A paixão pelo futebol pulsa forte no coração brasileiro. A cada Copa do Mundo, o grito de “Brasil! Olê, Olê, Olê!” ecoa por todo o país. Inspirado nessa energia contagiante e na busca incessante pela glória, o problema que você enfrentará agora é uma homenagem às nossas conquistas mais memoráveis no maior palco do futebol.

E foi baseado nesse grito que, na Copa do Mundo de 2018, foi criada uma música de mesmo nome. A música fala sobre as cinco vitórias da nossa seleção brasileira ao longo dos anos, como nos trechos:

“É, em cinquenta e oito, foi Pelé  
Em sessenta e dois, foi o Mané  
Em setenta, o esquadrão  
Primeiro a ser tricampeão

Oh, noventa e quatro, Romário  
Dois mil e dois, Fenômeno  
Primeiro tetracampeão  
Único penta é o Brasilzão!”

Na letra da música, “cinquenta e oito”, “sessenta e dois”, “setenta”, “noventa e quatro” e “dois mil e dois” se referem, respectivamente, aos anos 1958, 1962, 1970, 1994 e 2002, que correspondem aos cinco anos nos quais o Brasil venceu a Copa do Mundo.

Dado um ano específico, você deve determinar se o Brasil venceu a Copa do Mundo e levantou a taça naquele ano.

## Entrada

A entrada é composta por uma única linha contendo um único número inteiro  $N$ , representando o ano a ser verificado.

## Saída

Seu programa deverá imprimir uma única linha contendo um único caractere. Caso o Brasil tenha ganhado uma Copa do Mundo naquele ano, imprima o caractere **S** (a letra S maiúscula). Caso contrário, imprima o caractere **N** (a letra N maiúscula).

## Restrições

- $1930 \leq N \leq 2025$

## Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas restrições adicionais, definidas a seguir.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados na tarefa e serve para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (15 pontos):**  $N \geq 2010$ .
- **Subtarefa 3 (21 pontos):**  $N \geq 2000$ .
- **Subtarefa 4 (64 pontos):** Sem restrições adicionais.

**Exemplos**

|                                         |                                    |
|-----------------------------------------|------------------------------------|
| <b>Exemplo de entrada 1</b><br><br>1958 | <b>Exemplo de saída 1</b><br><br>S |
| <b>Exemplo de entrada 2</b><br><br>1982 | <b>Exemplo de saída 2</b><br><br>N |

## Doces da Estela

*Nome do arquivo:* `doces.c`, `doces.cpp`, `doces.pas`, `doces.java`, `doces.js` ou `doces.py`

Estela é uma grande doceira. Cada um dos pedidos feitos na sua confeitaria tem sempre  $N$  doces e, em cada pedido, todos eles precisam ser divididos em caixas com a mesma quantidade de doces, sem sobrar nenhum doce.

Uma caixa simples consiste de uma embalagem plástica e alguns doces (entre 1 e  $N$ ). A embalagem possui um preço fixo de 10 reais, enquanto cada doce adiciona 3 reais ao preço da caixa.

Além disso, os clientes não gostam muito de caixas simples, eles preferem caixas enfeitadas, cada uma com um lindo laço. Porém, Estela não gosta de colocar laços nas caixas porque isso dá muito trabalho. Então, quanto mais caixas enfeitadas tem em um pedido, mais Estela cobra por cada caixa. Em particular, em um pedido com  $C$  caixas enfeitadas o preço de cada caixa enfeitada é o preço de uma caixa simples multiplicado por  $(2+C)$ .

Por exemplo, para um pedido com 5 caixas enfeitadas com 2 doces em cada, Estela cobrará o valor de  $(2 + 5) \times (10 + 3 \times 2) = 112$  reais por caixa, totalizando  $112 \times 5 = 560$  reais pelo pedido.

Estela notou algo muito interessante sobre os pedidos de hoje: ela recebeu exatamente um pedido de caixas enfeitadas para cada divisão possível de  $N$  doces nas caixas. Uma divisão é possível se todos os doces são distribuídos igualmente nas caixas e não sobra nenhum. Por exemplo, se  $N = 10$ , ela receberia quatro pedidos: um pedido com uma caixa, outro com duas caixas, outro com cinco caixas e outro com dez caixas.

Sua tarefa é responder quantos reais Estela ganhará no total com os pedidos de hoje.

### Entrada

A única linha da entrada contém um inteiro  $N$ , que representa o número de doces em cada pedido.

### Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, o quanto Estela ganhará no total com os pedidos de hoje.

### Restrições

- $1 \leq N \leq 1\,000\,000$

**Para competidoras que utilizam C++ ou Java:** Observe que a resposta pode ser muito grande para caber em um inteiro de 32 bits. É recomendado o uso de um inteiro de 64 bits (`long long` em C++; `long` em Java). (*Competidoras usando Python ou JavaScript podem ignorar este aviso.*)

### Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas restrições adicionais às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (1 ponto):**  $N = 4$ .

- **Subtarefa 3 (3 pontos):**  $N \leq 5$ .
- **Subtarefa 4 (7 pontos):**  $N \leq 10$ .
- **Subtarefa 5 (26 pontos):**  $2 \leq N \leq 4999$  e  $N$  é primo. Em outras palavras, existem exatamente duas divisões possíveis: uma caixa com  $N$  doces ou  $N$  caixas com um doce.
- **Subtarefa 6 (29 pontos):**  $N \leq 1000$ .
- **Subtarefa 7 (34 pontos):** Sem restrições adicionais.

## Exemplos

|                                        |                                             |
|----------------------------------------|---------------------------------------------|
| <b>Exemplo de entrada 1</b><br>1       | <b>Exemplo de saída 1</b><br>39             |
| <b>Exemplo de entrada 2</b><br>5       | <b>Exemplo de saída 2</b><br>530            |
| <b>Exemplo de entrada 3</b><br>9       | <b>Exemplo de saída 3</b><br>1683           |
| <b>Exemplo de entrada 4</b><br>10      | <b>Exemplo de saída 4</b><br>2440           |
| <b>Exemplo de entrada 5</b><br>1000000 | <b>Exemplo de saída 5</b><br>21329695784850 |

## Fotografando Prédios

*Nome do arquivo:* foto.c, foto.cpp, foto.pas, foto.java, foto.js ou foto.py

Dhara está de férias, e decidiu chamar os amigos da rua em que ela mora para uma competição de fotografia. Na rua de Dhara, existem  $N$  prédios dispostos em uma linha, numerados de 1 a  $N$  da esquerda para a direita. Dhara tem em seu caderno anotadas as alturas de todos os prédios na lista  $A$ , onde  $A_i$  é a altura do prédio de número  $i$ .

Nessa competição, cada um dos  $N$  amigos de Dhara vai se posicionar em um prédio diferente e deve tirar fotos de prédios que estejam à direita (não é permitido tirar fotos do próprio prédio em que o fotógrafo está posicionado). Como alguns prédios podem ser maiores do que outros, pode não ser possível fotografar alguns prédios. Uma pessoa posicionada no prédio  $i$  só pode fotografar um prédio de número  $j$  se ele não for “tampado” por outro prédio de número  $k$ . Isso aconteceria se  $i < k < j$  e  $A_k \geq A_j$  (prédio à direita de  $i$  e à esquerda de  $j$  com altura maior ou superior que  $A_j$ ).

Como Dhara está organizando a competição, ela escreveu em seu caderno uma outra lista  $F$ , e decidiu que o amigo posicionado no prédio de número  $p$  deve tirar ao total exatamente  $F_p$  fotos, cada uma de um prédio diferente. As regras da competição dizem que a  $i$ -ésima foto tirada, considerando que o prédio de número  $k$  foi o fotografado, valerá  $i \cdot A_k$  pontos. Dessa forma, a pontuação de cada pessoa será a soma das pontuações de suas fotos, considerando a ordem em que foram tiradas.

Dhara está torcendo por cada um de seus amigos. A competição só vai ser na semana que vem, mas ela pediu que você já calculasse, a partir da lista  $A$  das alturas e da lista  $F$  das quantidades de fotos, qual seria a menor pontuação possível que cada amigo poderia atingir. É possível que Dhara tenha se confundido e alguns dos amigos não consigam tirar a quantidade de fotos exigida, e ela também deseja ser informada sobre isso.

### Entrada

A primeira linha da entrada contém o número inteiro  $N$ : a quantidade de amigos e de prédios na rua.

A segunda linha da entrada contém  $N$  inteiros: o  $i$ -ésimo da esquerda para a direita é  $A_i$  (a altura do prédio de número  $i$ ).

A terceira linha da entrada contém também  $N$  inteiros: o  $i$ -ésimo da esquerda para a direita é  $F_i$  (a quantidade de fotos que o amigo posicionado no prédio  $i$  deve tirar).

### Saída

Seu programa deve imprimir uma única linha contendo  $N$  inteiros: o  $i$ -ésimo da esquerda para a direita deve ser a menor pontuação possível que o amigo posicionado no prédio  $i$  pode atingir. Caso ele não consiga tirar  $F_i$  fotos, esse  $i$ -ésimo valor deve ser  $-1$  em vez disso.

### Restrições

- $1 \leq N \leq 100\,000$
- $1 \leq A \leq 100\,000\,000$
- $1 \leq F_i \leq N$

### Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (5 pontos):**  $N \leq 1000$  e  $A_i \leq 1000$ .
- **Subtarefa 3 (25 pontos):**  $F_i \leq 2$ .
- **Subtarefa 4 (70 pontos):** Sem restrições adicionais.

## Exemplos

|                                                                                              |                                                                     |
|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Exemplo de entrada 1</b><br><br>5<br>1 5 3 2 4<br>2 2 1 1 5                               | <b>Exemplo de saída 1</b><br><br>-1 10 2 4 -1                       |
| <b>Exemplo de entrada 2</b><br><br>10<br>18 94 72 18 40 1 56 78 91 24<br>5 2 5 4 3 1 2 6 2 7 | <b>Exemplo de saída 2</b><br><br>-1 222 665 575 193 56 247 -1 -1 -1 |



# Jogo do Castelo de Blocos

*Nome do arquivo:* `jogo.c`, `jogo.cpp`, `jogo.pas`, `jogo.java`, `jogo.js` ou `jogo.py`

Durante uma tarde chuvosa, os irmãos Ana e Bruno decidiram construir um imenso castelo de blocos na sala de estar. Cada bloco foi cuidadosamente posicionado de modo que alguns sirvam de “base” para sustentar outros. Podemos representar cada bloco com um número de 1 a  $N$ , e dizer que se o bloco  $u$  serve de base para sustentar o bloco  $v$ , então  $v$  se apoia diretamente no bloco  $u$ , formando uma relação de apoio entre os blocos. Se um bloco  $w$  se apoia diretamente em  $v$  e  $v$  se apoia diretamente em  $u$  dizemos que  $w$  se apoia indiretamente em  $u$ . A estrutura desse castelo era muito particular: um bloco nunca se apoiava direta ou indiretamente em si mesmo, então não haviam ciclos de apoios.

Depois de horas de trabalho, o castelo ficou tão alto que a mãe deles pediu: “Desmontem tudo antes do jantar, sem fazer bagunça!”

Como nenhum dos dois gosta de perder, eles transformaram a tarefa em um jogo:

Os irmãos Ana (Jogadora 1) e Bruno (Jogador 2) alternam turnos, com Ana sempre começando. Em cada turno, um jogador deve remover exatamente um bloco de sua escolha do castelo. O bloco é retirado cuidadosamente e pode ser guardado. No entanto, a remoção de um bloco tem consequências imediatas:

- Qualquer bloco que perca todos os blocos em que se apoiava **diretamente** no castelo desaba e é considerado “desmoronado”.
- A retirada ou desmoronamento de um bloco pode causar novos desmoronamentos em cascata, ou seja, se um bloco é desmoronado ou retirado, os blocos que dependiam dele também podem ficar sem suporte e, conseqüentemente, desmoronar.

O objetivo de Ana é minimizar o total de blocos desmoronados ao final do jogo, enquanto Bruno quer maximizar esse total. O jogo termina quando não restam mais blocos para serem removidos.

Considerando que ambos os irmãos jogam perfeitamente (ou seja, sempre fazem a melhor jogada possível para atingir seus objetivos), sua tarefa é determinar qual será o número de blocos que serão desmoronados ao final do jogo.

## Entrada

A primeira linha da entrada contém dois inteiros  $N$  e  $M$ , representando respectivamente o número total de blocos no castelo dos irmãos, numerados de 1 a  $N$ , e o número de relações de apoio entre os blocos.

As próximas  $M$  linhas contém dois inteiros  $u$  e  $v$ , indicando que o bloco  $v$  se apoia no bloco  $u$ .

## Saída

Seu programa deve imprimir um único inteiro: o número total de blocos que desmoronaram ao final de todas as operações.

## Restrições

- $1 \leq N \leq 18$
- $1 \leq M \leq N \times (N - 1)/2$

- As relações de apoio garantem que um bloco nunca se apoia em si mesmo, direta ou indiretamente, e não formam ciclos de dependência.

### Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas restrições adicionais às definidas acima.

- Subtarefa 1 (0 pontos): Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- Subtarefa 2 (18 pontos):  $v = u + 1$ , para todo par  $u, v$
- Subtarefa 3 (25 pontos):  $N \leq 10$
- Subtarefa 4 (57 pontos): Sem restrições adicionais.

### Exemplos

|                                                                    |                                    |
|--------------------------------------------------------------------|------------------------------------|
| <b>Exemplo de entrada 1</b><br><br>3 2<br>1 2<br>1 3               | <b>Exemplo de saída 1</b><br><br>1 |
| <b>Exemplo de entrada 2</b><br><br>5 4<br>1 2<br>1 3<br>2 4<br>3 5 | <b>Exemplo de saída 2</b><br><br>3 |

# Fila na Cantina

*Nome do arquivo:* `fila.c`, `fila.cpp`, `fila.pas`, `fila.java`, `fila.js` ou `fila.py`

Na escola municipal da Nlogópolis, o lanche mais esperado da cantina é o famoso croissant de chocolate. Diulia está muito interessada em estudar como a fila da cantina se comporta quando há croissants nos recreios.

Para isso, ela decidiu anotar em seu caderno a quantidade de pessoas na fila a cada minuto. A fila é bem dinâmica, então, a cada minuto, ou alguém entra ou sai da fila (nunca os dois ao mesmo tempo). Diulia começou a anotar assim que a primeira pessoa entrou na fila, e parou assim que a última pessoa saiu.

No final do dia, ela decidiu investigar os dados de suas anotações. Diulia escolheu  $Q$  “pedaços” para analisar, de forma que o  $i$ -ésimo pedaço corresponde aos minutos começando em  $c_i$  e terminando em  $t_i$  (incluindo os minutos  $c_i$  e  $t_i$ ). Ela garantiu que todos os pedaços são “válidos”, ou seja, a menor quantidade de pessoas entre  $c_i$  e  $t_i - 1$  nunca é menor do que a quantidade de pessoas na fila no minuto  $c_i$ , e, no minuto  $t_i$ , há a mesma quantidade de pessoas que no minuto  $c_i - 1$  (considere que há 0 pessoas no minuto 0).

Para cada um dos  $Q$  pedaços, Diulia deseja saber qual é a quantidade de pessoas na fila que mais se repetiu entre os minutos desse pedaço, e também quantas foram essas repetições.

## Entrada

A primeira linha da entrada contém dois inteiros  $N$  e  $Q$ , que representam a quantidade de minutos observados e a quantidade de pedaços que Diulia está interessada em investigar, respectivamente.

A próxima linha contém  $N$  inteiros separados por espaços em branco. O  $i$ -ésimo deles,  $P_i$ , representa a quantidade de pessoas no minuto  $i$ .

Cada uma das próximas  $Q$  linhas representa um pedaço que Diulia deseja investigar. A  $i$ -ésima dessas linhas contém dois inteiros  $c_i$  e  $t_i$ , representando os minutos correspondentes ao  $i$ -ésimo pedaço.

## Saída

Seu programa deverá imprimir  $Q$  linhas, cada uma contendo dois inteiros: na  $i$ -ésima linha, a quantidade de pessoas que mais se repetiu entre os minutos  $c_i$  e  $t_i$  e quantas foram essas repetições, respectivamente. Caso haja múltiplas quantidades empatadas com o maior número de repetições, o primeiro inteiro deverá ser a menor delas.

## Restrições

- $2 \leq N, Q \leq 600000$
- $P_i = P_{i-1} + 1$  ou  $P_i = P_{i-1} - 1$
- $P_1 = 1$  e  $P_N = 0$
- $0 \leq P_i \leq N$
- $1 \leq c_i < t_i \leq N$
- Os minutos de  $c_i$  a  $t_i$  formam um pedaço “válido”

## Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (7 pontos):**  $N \leq 1000$  e  $Q \leq 1000$ .
- **Subtarefa 3 (9 pontos):** no pedaço  $c = 1$  e  $t = N$ , nenhuma quantidade se repete mais de duas vezes.
- **Subtarefa 4 (18 pontos):**  $P_i \leq 30$ .
- **Subtarefa 5 (30 pontos):**  $N \leq 100000$  e  $Q \leq 100000$ .
- **Subtarefa 6 (36 pontos):** Sem restrições adicionais.

## Exemplos

|                                                                                                         |                                                                  |
|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| <b>Exemplo de entrada 1</b><br><br>4 2<br>1 2 1 0<br>1 4<br>2 3                                         | <b>Exemplo de saída 1</b><br><br>1 2<br>1 1                      |
| <b>Exemplo de entrada 2</b><br><br>12 5<br>1 0 1 2 3 4 3 4 3 2 1 0<br>1 2<br>5 10<br>3 12<br>6 7<br>8 9 | <b>Exemplo de saída 2</b><br><br>0 1<br>3 3<br>3 3<br>3 1<br>3 1 |